



RAPPORT FINAL

Un Outil d'Aide pour la Planification des Examens

Garrec Aimie
Fernandes Valentin
Droin Charlotte
Lecoeur Bertrand
Théret Sarah

Sommaire

Introduction.....	3
I. Recherches bibliographiques.....	3
A. Coloration de graphe.....	3
a. Algorithme de Welsh et Powell.....	4
b. Algorithme DSatur.....	6
c. RLF - Recursive Largest First.....	7
B. Génétique.....	9
II. Notre travail.....	11
A. Lien Github.....	11
B. L'organisation.....	11
a. Les outils de communication.....	11
b. Répartition du travail et gestion du temps.....	11
c. Points forts et points faibles de l'équipe.....	12
C. L'application de gestion des examens.....	12
a. Cahier des charges.....	12
b. La réflexion au préalable.....	15
c. Jeu de données.....	15
d. La solution conceptuelle et le code.....	17
e. L'interface.....	21
Conclusion.....	23
Bibliographie.....	24

Introduction

L'optimisation d'emploi du temps d'examen est une question complexe à traiter pour les établissements scolaires. La quantité de paramètres à prendre en compte pour garantir la qualité des solutions optimales proposées rendent la tâche d'autant plus difficile. Il s'agit d'allier les préférences individuelles des étudiants, des professeurs encadrants et les contraintes d'espaces et de temps immuables. C'est pourquoi nous devons donner un ordre de préférence quant au respect de ces contraintes en les hiérarchisant. D'où la séparation entre les contraintes "dures" et "molles".

L'approche la plus naturelle pour aborder un tel problème serait de calculer toutes les combinaisons possibles d'emploi du temps envisageable et de sélectionner le meilleur. Cette méthode conceptuellement acceptable se heurte dans la pratique à la limitation de la complexité spatiale et temporelle imposée par nos ordinateurs.

L'enjeu de nos recherches est alors de déterminer une solution acceptable, à défaut d'être absolument optimale, au recours de méthodes numériques et mathématiques ayant une complexité raisonnable.

Nous présenterons dans un premier temps, un état de l'art, c'est-à-dire les solutions existantes que nous jugeons les plus pertinentes. Nous les avons séparés en deux types : les méthodes de coloration de graphe et les approches plus heuristiques. Ensuite, nous expliquerons le travail que l'on a fourni qui consiste en un cahier des charges, une réflexion sur les outils et méthodes que nous allons utiliser, l'analyse de notre code et enfin des visualisations de l'interface.

I. Recherches bibliographiques

A. Coloration de graphe

Pour résoudre le problème de planification d'examens, on peut utiliser une méthode de théorie des graphes nommée la coloration de graphe.

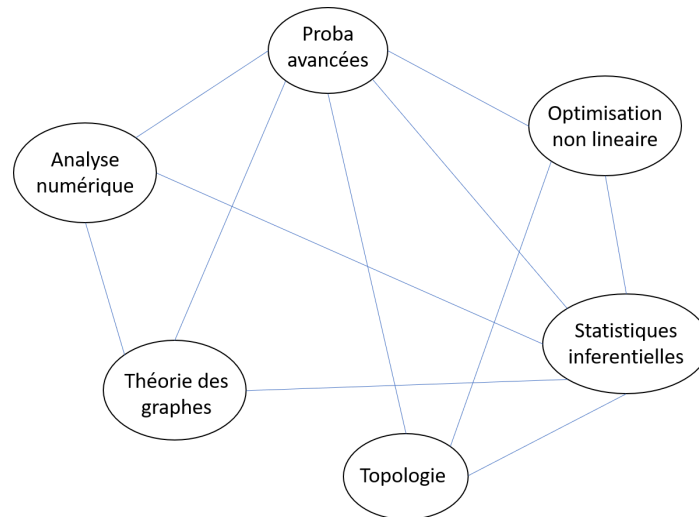
Pour cela, nous allons représenter les examens sous forme de sommets dans un graphe où les arêtes représentent les conflits d'horaires entre les examens, c'est-à-dire que s'il existe une arête entre deux examens, alors au moins un élève doit passer ces deux examens donc ils ne peuvent pas se dérouler en même temps. Ensuite, les sommets sont colorés de manière à ce que les sommets adjacents (représentant des examens conflictuels) aient des couleurs différentes. Le but de cette méthode est de minimiser le nombre de couleurs utilisées, c'est-à-dire de minimiser le nombre de sessions d'examen nécessaires.

Pour les différentes méthodes que nous allons étudier, nous allons utiliser le même exemple contenant deux classes de première année : GMI et GMF.

Les GMI ont comme matières : Proba avancées, Statistiques inferentielles, Analyse numérique, Théorie des graphes.

Et que les GMF ont comme matière : Proba avancées, Statistiques inferentielles, Optimisation non linéaire, Topologie.

En reliant les matières qui ne peuvent pas avoir lieu en même temps, on obtient le graphe suivant :



a. Algorithme de Welsh et Powell

L'algorithme de Welsh-Powell [1] est un algorithme de type glouton introduit en 1967, c'est l'une des méthodes classiques pour résoudre un problème de coloration de graphe. Si on prend n le nombre de sommets et m le nombre d'arêtes du graphe, la complexité de cet algorithme est $O(n \cdot \log(n) + m)$.

En triant les degrés des sommets dans l'ordre décroissant, on obtient le tableau suivant :

matière	Proba avancées	Statistiques inferentielles	Analyse numérique	Théorie des graphes	Topologie	Optimisation non linéaire
degré	5	5	3	3	3	3

Ensuite on colore les sommets par degrés décroissants. On commence par Proba avancées, qu'on colore en jaune. On regarde ensuite quels autres sommets peuvent être coloriés en jaune sans rentrer en conflit, c'est-à-dire quels sommets n'a pas d'arêtes avec proba avancées. On les traite par ordre décroissant à nouveau. On regarde alors Statistiques inférentielles puis Analyse numérique puis Théorie des graphes puis Topologie

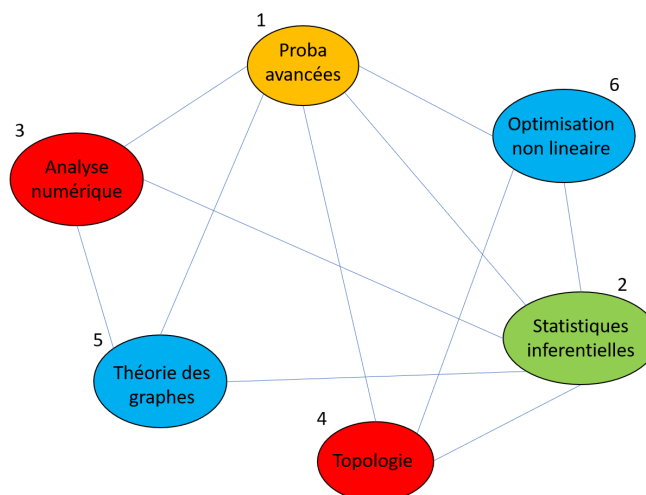
puis Optimisation non linéaire mais tous ont une arête avec proba Avancées donc on passe à la couleur suivante.

On colorie Statistiques inférentielles en vert et on regarde Analyse numérique puis Théorie des graphes puis Topologie puis Optimisation non linéaire mais tous ont une arête avec Statistiques inférentielles donc on passe à la couleur suivante.

On colorie Analyse numérique en rouge puis on regarde Théorie des graphes qui a une arête avec Analyse numérique. On regarde alors Topologie qui n'a pas d'arête avec Analyse numérique donc on colorie Topologie en rouge. On regarde finalement Optimisation non linéaire qui n'a pas d'arête avec Analyse numérique mais en a une avec Topologie donc on passe à la couleur suivante.

On colorie Théorie des graphes en bleu puis on regarde Optimisation non linéaire qui est le seul sommet non colorié. Il n'a pas d'arête avec Théorie des graphes donc on le colorie en bleu. Tous les sommets ont été coloriés, on peut s'arrêter.

On obtient ce graphe final coloré où les numéro correspondent à l'ordre de coloration :



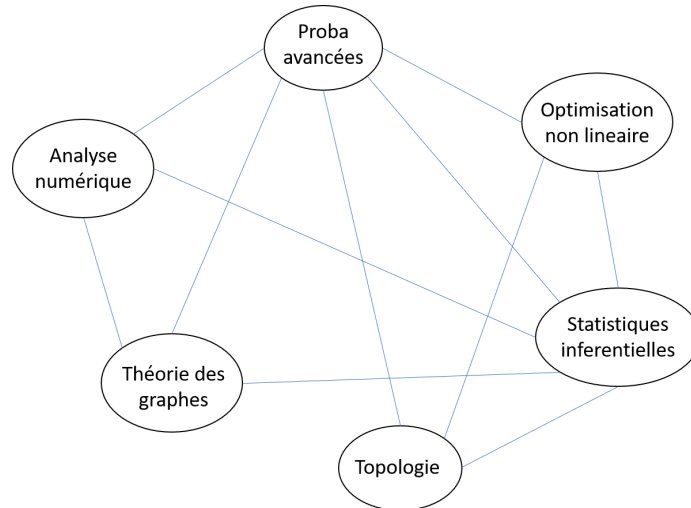
Ici nous avons utilisé 4 couleurs, et il est maintenant possible de dire que nous pouvons faire passer aux élèves de GMI l'examen d'Analyse numérique à la même heure que l'examen de Topologie des GMF, de même pour l'Optimisation non linéaire des GMF et la Théorie des graphes des GMI, enfin les Proba avancées et Statistiques inférentielles devront être des examens individuels à des heures différentes.

Les avantages de cet algorithme sont qu'il est simple, rapide à implémenter et efficace pour les petits graphes ou les graphes de structure simple. Néanmoins, cet algorithme ne garantit pas une solution optimale c'est-à-dire l'utilisation du nombre minimum de couleurs.

b. Algorithme DSatur

L'algorithme Dsatur [2] est un algorithme heuristique introduit en 1979. Si n est le nombre de sommets, la complexité de cet algorithme est $O(n^2)$ pour des graphes complexes et $O(n \cdot \log(n))$ dans le meilleur des cas.

On rappelle le graphe sur lequel nous faisons l'exemple :



On doit d'abord faire une liste de couleurs avec un ordre de priorité. Ici nous allons prendre :

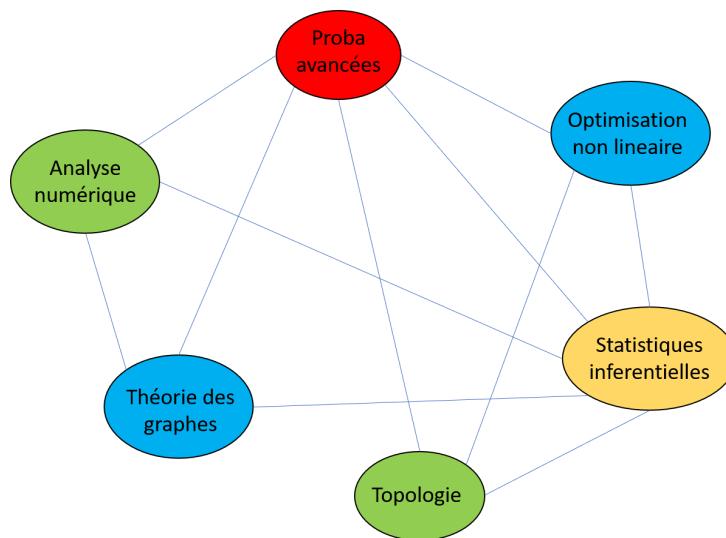
1-Rouge 2-Jaune 3-Vert 4-Bleu

On part d'un des sommets avec le degré le plus élevé, ici on peut prendre Proba avancées que l'on va colorier en rouge. On va ensuite colorier les sommets un à un par ordre décroissant de degré de saturation (=nombre de voisins colorés). En cas d'égalité entre plusieurs sommets ayant le même degré de saturation, on choisira le sommet avec le degré (=nombre de voisins) le plus élevé. Ensuite, pour choisir la couleur, on choisira la couleur la plus basse possible c'est-à-dire le rouge puis jaune puis vert puis bleu qui n'est pas utilisée par les voisins du sommet étudié.

Sommet	Proba avancées	Analyse numérique	Théorie des graphes	Topologie	Statistiques inferentielles	Optimisation non linéaire
étape 1	Rouge	Dsat=1 degré=3	Dsat=1 degré=3	Dsat=1 degré=3	Dsat=1 degré=5	Dsat=1 degré=3
étape 2	Rouge	Dsat=2 degré=3	Dsat=2 degré=3	Dsat=2 degré=3	Jaune	Dsat=2 degré=3
étape 3	Rouge	Vert	Dsat=3 degré=3	Dsat=2 degré=3	Jaune	Dsat=2 degré=3

étape 4	Rouge	Vert	Bleu	Dsat=2 degré=3	Jaune	Dsat=2 degré=3
étape 5	Rouge	Vert	Bleu	Vert	Jaune	Dsat=3 degré=3
étape 6	Rouge	Vert	Bleu	Vert	Jaune	Bleu

On obtient donc le graphe coloré suivant :

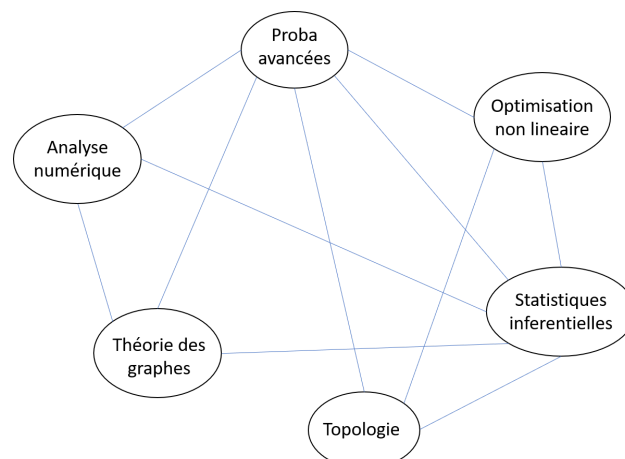


L'avantage de cet algorithme est qu'il peut être appliqué à des graphes complexes ou irréguliers. Par conséquent, l'algorithme Dsat est plus difficile à implémenter que l'algorithme de Welsh et Powell.

c. RLF - Recursive Largest First

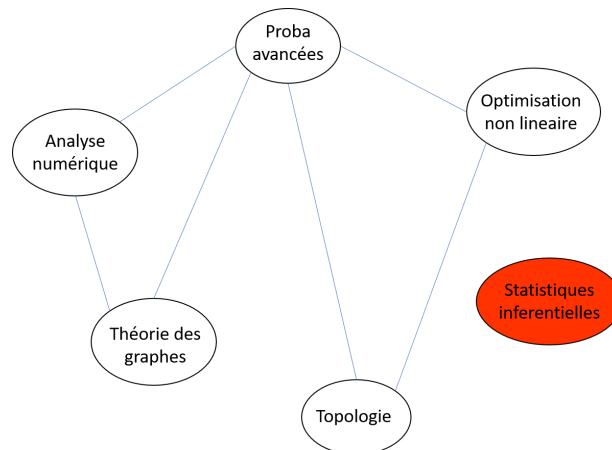
L'algorithme Recursive Largest First [3] est un algorithme heuristique proposé en 1979 de complexité $O(n^3)$.

On rappelle le graphe sur lequel nous faisons l'exemple :



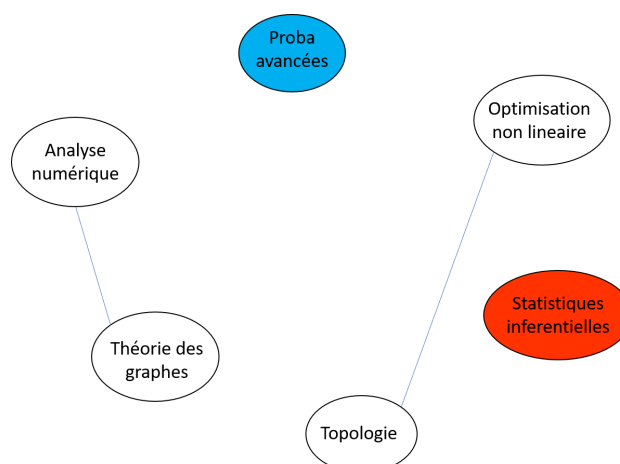
Cet algorithme construit des ensembles indépendants de sommets c'est-à-dire des ensembles de sommets qui ne sont pas adjacents les uns aux autres, qu'on nommera S . Puis colorie chacun de ces ensembles d'une couleur différente.

On commence par colorier le sommets avec le degré le plus élevé, ici Statistiques inférentielles (comme il y en a deux on choisit arbitrairement). On le colorie en rouge et on l'ajoute au premier ensemble indépendant de sommets S_1 , puis on regarde les sommets non colorés qui ne sont pas adjacents à Statistiques inférentielles, sauf qu'il n'y en a aucun donc $S_1 = \{\text{Statistiques inférentielles}\}$. On supprime ensuite toutes les arêtes reliées à Statistiques inférentielles, afin d'obtenir le sous-graphe suivant :



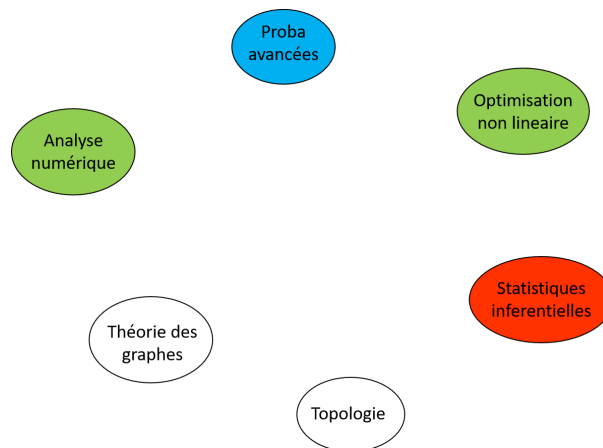
On passe au deuxième ensemble indépendant de sommets S_2 , maintenant le sommet avec le degré le plus élevé est Proba avancées, qu'on colorie donc en bleu et ajoutons à S_2 . Aucun sommet non colorés n'est pas un voisin de Proba avancées, donc on a $S_2 = \{\text{Proba avancées}\}$.

On supprime ensuite toutes les arêtes reliées à Proba avancées, afin d'obtenir le sous-graphe suivant :

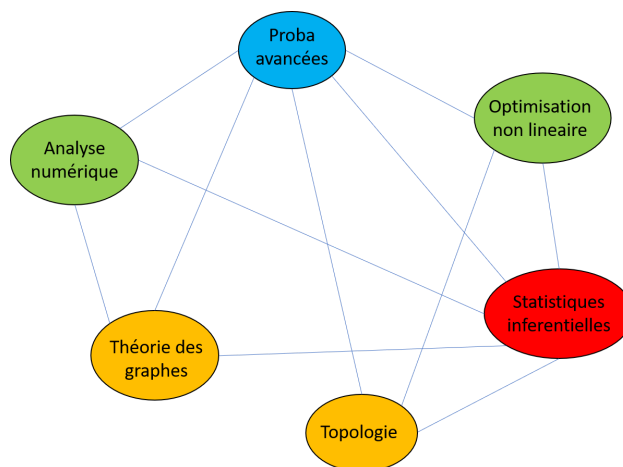


On passe maintenant au troisième ensemble indépendant de sommets S_3 avec Optimisation non linéaire comme sommet ayant le plus grand degré, qu'on colorie en vert et que l'on ajoute à S_3 . On regarde ensuite les sommets non colorés et non voisins d'Optimisation linéaire et on trouve Analyse numérique qu'on ajoute à S_3 . Après chaque ajout de sommet dans S_3 on vérifie s'il y a encore des sommets non colorés et non voisins de

l'ensemble des sommets de S_3 . Ici il n'y en a plus donc on a $S_3 = \{\text{Optimisation linéaire, Analyse numérique}\}$. On supprime ensuite toutes les arêtes reliées à chaque sommets de S_3 , afin d'obtenir le sous-graphe suivant :



Enfin, lorsqu'il n'y a plus d'arête dans le graphe, on peut ajouter tous les sommets restant dans l'ensemble S_4 et les colorier en jaune. On a donc $S_4 = \{\text{Théorie des graphes, Topologie}\}$ et on peut conclure que l'ensembles indépendants de sommets $S = \{\{\text{Statistiques inferentielles}\}, \{\text{Proba avancées}\}, \{\text{Optimisation linéaire, Analyse numérique}\}, \{\text{Théorie des graphes, Topologie}\}\}$ avec le graphe coloré suivant :



Les inconvénients de cet algorithme sont qu'il est plus difficile à implémenter que les deux algorithmes précédents et est plus lent pour les grands graphes ou les graphes denses.

B. Génétique

Voyons à présent une approche heuristique de la planification d'examen. Plus particulièrement, voici une méthode méta-heuristique basée sur un algorithme génétique. [4]

Le caractère "génétique" de cet algorithme repose sur la sélection naturelle et la mutation génétique aboutissant à une solution. L'aspect méta-heuristique de cette méthode est dû à la création d'une fonction objective que l'on cherche à minimiser.

Nous appellerons par la suite “chromosome” une liste de gènes. Et nous entendons par “gène” le regroupement d’information concernant une session d’examen.

Voici un exemple de gène : Topologie, 9h30 - 11h30, surveillé par Fred, en Hélix 2, promo ING1, etc

Dans l’étude “*a complete solution for planning scheduling*”, il y a 14 paramètres pour un gène afin de prendre en compte toutes les composantes essentielles à la qualification d’une session d’examen. Autrement dit, un chromosome est assimilable à un emploi du temps d’examen.

Par la suite, il s’agit de savoir si un chromosome constitue un bon ou très mauvais emploi du temps. Pour cela, on donne des pénalités pour chaque contrainte que l’emploi du temps ne respecte pas. La pénalité est d’autant plus élevée que la contrainte est dure. Par exemple : le professeur SIMON ne voulait pas surveiller l’examen de Topo le matin à cause d’un empêchement personnel, donc on donne une pénalité de 1 point (contrainte douce). On peut aussi considérer que l’examen de topologie et de proba avancée tombe le même jour, à la même heure, on donne alors une pénalité de 10 points (overlapping : contrainte dur).

Enfin, on additionne les pénalités, le résultat de cette somme est la valeur de $F(x)$ (où F est la fonction objectif, donnant le score de pénalité d’un chromosome x). Dans notre exemple, on a que $F(x) = 10 + 1 = 11$. (valeurs données arbitrairement pour l’exemple)

Ainsi, le but de cet algorithme est de trouver le chromosome avec le score de pénalité le plus faible. On cherche le chromosome qui minimise F .

Une fois que le problème d’optimisation a été décomposé comme expliqué précédemment, il s’agit de comprendre le fonctionnement général de l’algorithme génétique.

Dans un premier temps, nous choisissons au hasard 100 chromosomes (le nombre de chromosomes à créer est choisi arbitrairement pour faciliter l’explication de la méthode). Le nombre de gènes constituant un chromosome est à calculer en fonction du nombre de matières notamment.

Ensuite nous répétons N fois les deux étapes suivantes. Étape 1 : On choisit les chromosomes avec les scores de pénalités les plus faibles de la génération précédente. Par exemple, on choisit (emploiDuTempsVersion1 avec $F=11$) et (emploiDuTempsVersion2 avec $F=9$). Étape 2 : On crée 100 chromosomes qui sont des versions légèrement modifiées des meilleurs chromosomes de la génération précédente sélectionnés à l’étape 1. Par exemple : On crée un chromosome qui déplace l’examen de topologie à 14h au lieu de 9h30 pour son chromosome père, tout en gardant le reste des gènes identique.

On choisit à l’issue des N itérations, le meilleur chromosome de la dernière génération, qui sera alors le meilleur emploi du temps que l’on ait pu déterminer.

Cet algorithme prend en compte les contraintes dures et molles. De plus, le résultat est directement exploitable car il s’agit d’un chromosome dont on peut directement déduire un emploi du temps. La complexité est maîtrisée par notre contrôle sur l’algorithme génétique. Malgré tout, cette solution demande plus de temps de développement, ce qui est à prendre en compte lors de la sélection de notre approche de prédilection.

II. Notre travail

A. Lien Github

[ValentinLaTortue/Projet-Planning- \(github.com\)](https://github.com/ValentinLaTortue/Projet-Planning-)

B. L'organisation

a. Les outils de communication

Les premiers outils mis en place par l'équipe ont été un groupe WhatsApp et un drive partagé. Ils ont été choisis pour leur simplicité d'utilisation et car tous les membres de l'équipe les maîtrisaient. Mais dès le début du projet, nous nous sommes rapidement aperçus que ces outils ne suffiraient pas. Un groupe Discord a alors été créé, ce qui nous a permis de pouvoir partager nos écrans lors de nos réunions. Puis lorsque nous avons commencé à coder, nous avons finalement créé notre GitHub. Ce dernier outil était nouveau pour l'ensemble du groupe ce qui nous a légèrement déstabilisé notamment lors de la configuration mais nous avons fini par tous réussir à le prendre en main.

b. Répartition du travail et gestion du temps

Tous les deux jours environ, en fonction de nos disponibilités personnelles, nous nous appelions afin de faire un point sur notre avancement. C'est lors de ces réunions que l'on répartissait les tâches. Cette répartition s'est faite assez naturellement selon les intérêts de chacun. Ainsi, pour le premier rendu, nous avons d'abord décidé de tous participer aux recherches bibliographiques pour bien comprendre le sujet. Puis nous avons chacun développé nos recherches sur un algorithme qui nous plaisait pour pouvoir l'expliquer aux autres. Lorsque les bases du sujet et le fonctionnement des différents algorithmes ont été compris de tous, nous avons choisi ensemble lequel nous souhaitions implémenter. Nous avons ensuite divisé le code en 4 parties distinctes : la création du jeu de données et sa lecture, l'algorithme pur (ici DSatur), l'assignation des créneaux et des salles et enfin l'interface. Nous avons initialement assigné une personne par tâche sauf pour l'algorithme où 2 personnes en étaient responsables. Mais au fur et à mesure, certaines personnes ont réalisé leurs tâches plus rapidement donc elles ont aidé les autres sur les tâches plus ardues. De même, nous nous étions fixés comme délai d'avoir fini le code sans l'interface pour le jeudi 23 mai. Ce délai a été respecté dans le sens où chacun avait fini sa partie de code mais lors de la fusion des codes, nous avons rencontré quelques problèmes. Nous avons remarqué que certaines parties du code renvoyaient des résultats erronés, nous avons donc finalisé le code samedi 25 mai.

c. Points forts et points faibles de l'équipe

On a créé notre équipe de travail à partir de notre groupe d'amis, grâce à cela on a une bonne cohésion de groupe, on se fait confiance et durant les réunions on avance efficacement. De plus, on a chacun et chacune des compétences qui nous sont propres, certains maîtrisent mieux la gestion de données tandis que d'autres maîtrisent mieux l'utilisation d'interface ou de python. Néanmoins, le seul point négatif que l'on a rencontré est la gestion du temps. En effet, certaines périodes pouvaient être qualifiées de très "productives" tandis qu'il y avait des jours où notre projet avançait un peu moins. De plus, plusieurs fois nous pensions aboutir à un code "final" mais finalement ce code devait être souvent modifié afin de répondre aux nouvelles contraintes que nous nous imposions. C'est aussi de cette manière que nous avons abouti à un code qui nous convenait de plus en plus. Finalement, le code que nous avons obtenu nous a globalement tous satisfaits mais nous nous sommes bien rendus compte qu'il aurait pu encore être amélioré si nous avions eu plus de temps.

C. L'application de gestion des examens

a. Cahier des charges

Introduction :

Le projet que nous avons choisi s'appelle "Un Outil d'Aide pour la Planification des Examens". Cela consiste à optimiser les horaires d'examens d'une école, d'une université ou d'un quelconque établissement, tout en prenant en compte certaines contraintes. Nous avons choisi ce problème car il nous semblait intéressant, complexe et challengeant. De plus, étant étudiant avec de nombreuses matières, c'est une question que nous nous sommes déjà posée. Vous trouverez ainsi dans ce cahier des charges le contexte du projet, le besoin ainsi que les résultats attendus, mais également les contraintes molles et dures du problème. Nous listerons aussi les ressources nécessaires, et fixerons les délais clairs et réalistes du projet.

Présenter le contexte du projet :

Ce projet vise à développer un outil informatique destiné à faciliter la planification des examens pour les étudiants. Actuellement, la planification manuelle des examens devient de plus en plus complexe en raison de l'augmentation du nombre d'étudiants et de matières. L'objectif principal est d'automatiser ce processus afin d'économiser du temps et de garantir une planification efficace. Mais planifier des examens de façon efficace peut être compliqué. Ainsi, la résolution de ce problème de planification d'examens est considérée comme un défi d'optimisation combinatoire complexe. Nous verrons dans ce rapport que certaines méthodes utilisées sont efficaces pour de petits plannings, mais moins adaptées aux plannings complexes.

Définir le besoin avec précision :

Le besoin précis de ce projet est de développer un algorithme capable de prendre en compte toutes ces contraintes et de générer des plannings d'examens efficaces et optimisés. Ce besoin est de plus en plus nécessaire pour de nombreux établissements car le nombre d'étudiants est en constante augmentation chaque année, et donc les besoins et les contraintes changent et évoluent avec, obligeant les établissements à constamment s'adapter.

Identifier les résultats attendus :

Les résultats attendus sont divers et nombreux :

Tout d'abord, l'application devrait aider l'administration dans la planification des examens. Nous allons devoir tenir compte des contraintes dures et molles. Les contraintes molles seront définies par nous-mêmes dans la suite du projet. L'application devra également être capable de détecter et d'éviter les conflits d'horaires pour les étudiants inscrits simultanément à plusieurs matières, assurant ainsi qu'ils puissent passer tous leurs examens sans problème. Enfin, l'application devra être robuste et flexible, capable de gérer différentes configurations et contraintes spécifiques à chaque session d'examen, tout en fournissant des plannings d'examen fiables et adaptés aux besoins de l'institution.

Cadrer les contraintes du projet :

Les contraintes dures sont immuables et déjà définies dans les attendus du projet, tandis que les contraintes molles dépendent de nous. Ainsi, les contraintes dures doivent être obligatoirement respectées, au contraire des contraintes molles que nous pourrions ajuster et sélectionner à notre guise.

Contraintes dures :

- Un élève ne peut pas être dans 2 épreuves en même temps.
- Il doit y avoir suffisamment de salles pour chaque session.
- Il doit être possible d'avoir plusieurs promotions dans une même session d'une épreuve.

Contraintes douces:

- Mettre un temps de pause de 1h entre chaque examen et de 12h à 13h.
- Ne pas mettre plus de 4 examens par journée, entre 9h et 18h.
- Regrouper par journées des matières du même module.
- Un examen peut se dérouler dans plusieurs salles.
- Un examinateur ne peut pas être dans 2 salles différentes en même temps.
- La gestion et répartition des élèves ayant besoin d'un tiers-temps.

Liste des ressources nécessaires:

Tout d'abord, nous avons besoin des références bibliographiques données afin de prendre connaissance du sujet et de nous aider pour l'élaboration de notre méthode envisagée. Par exemple:

- 1) Carter, M.W., & Laporte, G. (1996). "Recent Developments in Practical Examination Timetabling". In Interfaces, 26(6), 1-15.
- 2) Lewis, R., & Paechter, B. (2014). "A Survey of Metaheuristic Techniques for Single Machine Scheduling". In Journal of Heuristics, 20(3), 353-377.
- 3) Ross, P. (2010). "Handbook of Automated Essay Evaluation: Current Applications and New Directions". Routledge.

Nous avons également besoin de références complémentaires, notamment des thèses sur le sujet, des articles, mais également des vidéos sur youtube qui traitent de ce sujet. Cela sera développé plus en détail dans la synthèse de recherches.

Ensuite, nous avons besoin de trouver des façons de communiquer pour le groupe du projet. Nous avons décidé de créer un groupe Whatsapp pour écrire des messages et un groupe Discord pour les appels de groupe. Nous avons aussi un drive partagé où nous pouvons déposer nos documents et nos travaux.

Enfin, il faut que nous créions un github afin de pouvoir partager notre code.

Fixer des délais clairs et réalistes:

- Nous nous appelons tous les 2 jours sur discord afin de faire un point sur nos avancées respectives. En effet, nous répartissons le travail et nos appels nous permettent de parler des points importants pour l'avancée du projet.
- L'école nous impose deux rendus obligatoires. Le rendu 1 étant le 19 mai et le rendu 2 le 26 mai. Le rendu 1 comprend la synthèse des travaux existants, le cahier des charges et une solution conceptuelle du projet. Le rendu 2, quant à lui, doit contenir un jeu de données complet, une explication de notre choix de langage de programmation, un UML et un lien github pour avoir accès à notre implémentation complète.
- De notre côté, nous souhaitons terminer le rendu 1 le 18 mai, afin de pouvoir commencer au plus tôt l'implémentation du code, et donc par la même occasion le rendu 2. On aura donc eu plus d'une semaine pour faire des recherches approfondies sur le sujet, et 1 semaine pour la maquette de l'application future sous forme d'UML et construire un jeu de données complet.

Conclusion:

En suivant les étapes du cahier des charges, nous serons en mesure d'achever ce projet dans les temps. Cependant, le ressenti de tous les membres du groupe est la complexité de ce projet, et le temps restreint qui nous est donné. Mais le travail a bien été réparti et permettra de le réaliser efficacement grâce à une motivation et une implication présente chez tous les membres du groupe.

b. La réflexion au préalable

Avant de commencer le code, nous avons dû nous mettre d'accord sur le langage et l'algorithme que nous allions utiliser. Nous avons tout d'abord étudié les avantages et inconvénients de chacun des langages proposés dans le sujet, à savoir Python, C, C++ et Java. Le langage C++ a directement été exclu puisqu'aucun d'entre nous n'avait des connaissances dessus. Quant à C, bien que ce soit le plus performant en termes de puissance de calcul, ce n'est pas un langage orienté objet, il demande un niveau de maîtrise assez élevé en programmation et les librairies sont plus difficiles à utiliser et possèdent moins de fonctionnalités par défaut. Nous avons donc décidé de l'exclure également. A ce stade, il nous restait donc à choisir entre Python et Java. Python offre une syntaxe simple, un typage dynamique, un garbage collector et bien d'autres aides. Il est très flexible grâce aux larges ressources communautaires et permet de coder très rapidement. Il est donc idéal pour du prototypage de solution. Mais l'exécution des calculs est 10 à 20 fois plus lente que les trois autres langages, il est donc inadapté pour les grands calculs. Java, lui, calcule quasiment aussi vite que le C et le C++ et offre autant de bibliothèques que Python. Il aurait pu être judicieux de choisir Java pour ce projet. Pour autant, nous avons décidé d'utiliser Python car nous nous sentions plus à l'aise avec ce langage. En effet, nous avions tous minimum deux années d'expérience avec Python alors que la majorité d'entre nous venaient de découvrir Java il y a moins de 6 mois. Nous ne voulions pas nous rajouter cette contrainte et avons préféré nous concentrer sur le projet en lui-même.

Quant à la méthode, nous avons été attiré par la méthode génétique qui permettait de mieux gérer les contraintes molles et qui offrait une grande flexibilité à l'utilisateur puisque qu'il pouvait choisir l'importance d'une contrainte. Mais c'est une méthode très complexe et le sujet nous conseillait de partir sur une méthode utilisant la coloration de graphe. Nous avons donc comparé les complexités temporelles de l'algorithme de Welsh et Powell, de l'algorithme DSatur et de l'algorithme RLF. La meilleure complexité était donc celle de l'algorithme de Welsh et Powell ($O(n \cdot \log(n) + m)$). Toutefois, lors de nos recherches bibliographiques, nous nous sommes rendus compte que plusieurs sources décrivaient l'algorithme DSatur comme plus adapté aux graphes complexes et irréguliers. Pour s'assurer que nous faisions le bon choix, nous avons réalisé les pseudo-codes de ces deux algorithmes avec des exemples de leur déroulement. Nous avons finalement conclu que nous étions capables de coder ces deux méthodes et donc que notre choix se porterait sur DSatur.

c. Jeu de données

Nous avons travaillé avec 3 fichiers csv : Promotions.csv, Matières.csv, Salles.csv.

Tout d'abord, nous avons pris comme type d'école, une école d'art composée de 5 promotions : Sculpture, Création, Dessin, Peinture, Design.

Tronc commun : Technique de dessin, Théorie des formes et couleurs, Histoire de l'art, Art numérique

Création -> Céramique, Mosaïque, Etude des matériaux, Design de produit

Peinture -> Croquis, Peinture à huile, Peinture acrylique, Peinture contemporaine

Design -> Illustration, Graphisme, Design de produit, Typographie

B	C
elevés	liste_matières
20	Technique_dessin;Theorie_forme_couleurs;Histoire_art;Art_numerique;Etude_materiaux;Modelage;Sculpture_bois_pierre;Dessin_observation
25	Technique_dessin;Theorie_forme_couleurs;Histoire_art;Art_numerique;Ceramique;Mosaïque;Etude_materiaux;Design_produit
40	Technique_dessin;Theorie_forme_couleurs;Histoire_art;Art_numerique;Dessin_observation;Croquis;Illustration;Dessin_anatomique
30	Technique_dessin;Theorie_forme_couleurs;Histoire_art;Art_numerique;Croquis;Peinture_huile;Peinture_acrylique;Peinture_contemporaine
55	Technique_dessin;Theorie_forme_couleurs;Histoire_art;Art_numerique;Illustration;Graphisme;Design_produit;Typographie

Voici Matières.csv :

	A	B
1	nom	duree
2	Technique_dessin	2
3	Theorie_forme_couleurs	3
4	Histoire_art	4
5	Art_numerique	2
6	Etude_materiaux	3
7	Modelage	3
8	Sculpture_bois_pierre	4
9	Dessin_observation	2
10	Ceramique	2
11	Mosaïque	3
12	Design_produit	2
13	Croquis	3
14	Illustration	2
15	Dessin_anatomique	2
16	Peinture_huile	3
17	Peinture_acrylique	2
18	Peinture_contemporaine	3
19	Graphisme	2
20	Typographie	2

[illegible]

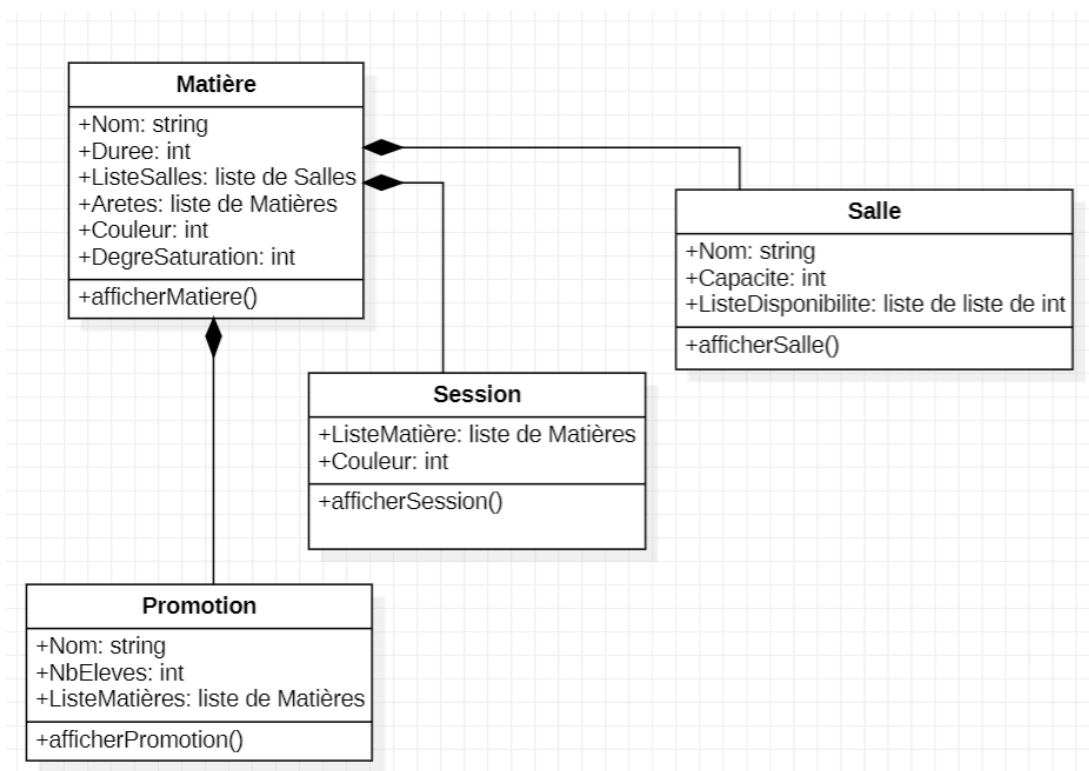
Pour chaque salle est associée une liste de disponibilité, chaque sous-liste représente un jour différent, par exemple si on nomme L la liste de disponibilité de la salle1 : $L[0]=[1,1,1,1,1,1,1,1,1,1,0,0,0,0,0,0,0]$, chaque chiffre correspond à un créneau d'une demi-heure, 1 signifie que la salle est occupé et 0 signifie que la salle est libre. Cette liste recouvre la plage horaire de 8h30 à 17h30, comme on peut le voir la salle est donc occupée de 8h30 à 14h et est libre de 14h à 17h30.

d. La solution conceptuelle et le code

Pour le code, nous partions du pseudo code de DSatur suivant :

- 1) *On appelle v le sommet avec le plus haut degré de saturation*
(Si plusieurs sommet ont le plus haut degré de saturation, on choisit celui avec le plus d'arêtes)
- 2) *On donne à v la plus basse couleur possible, qui n'est bien entendu pas encore utilisé par ses voisins*
- 3) *Si tous les sommets ont été colorié, on a fini et on retourne le graphe colorié, Sinon on retourne à l'étape 1).*

Mais pour pouvoir l'appliquer, il fallait savoir quelle était notre définition des sommets. Nous avons donc décidé d'une approche par classe. Ainsi nous avons construit un UML :



Les sommets sont donc les matières et les arêtes des matières sont remplis par un parcours des promotions.

Pour implémenter DSatur, nous avons rencontré quelques difficultés.

Tout d'abord il a fallu déterminer comment différencier les sommets déjà traités des sommets non traités. Pour cela, nous avons convenu que les couleurs des sommets seraient initialisées à 0. Ainsi, si la couleur des sommets est différente de 0, il a déjà été traité.

Ensuite, il fallait trouver une solution pour gérer la couleur. En effet, on fixe initialement la couleur à 1 puis on parcourt ses voisins pour vérifier qu'il n'y en a aucun de la couleur 1. Si c'est le cas, alors on teste la couleur 2. Mais si on suppose qu'il y a n sommets, et que le sommet testé a $n-1$ voisins, tous d'une couleur différente comprise entre 1 et $n-1$, alors le sommet testé devra prendre la couleur n . Pour le déterminer, on pourrait réaliser n parcours de la liste des voisins de taille $n-1$ pour tester chaque couleur une par une. Mais rien que cette étape aurait une complexité dans le pire des cas de $O(n^2)$. Cela nous semblait beaucoup et coûteux donc nous avons réfléchi à comment optimiser. C'est alors que nous avons pensé à un tableau qui ressemblerait à un histogramme (sans compter les effectifs puisque cela nous importe peu mais seulement en marquant la présence par un 1). C'est un tableau de taille n et lors du premier parcours des voisins, dès que l'on rencontre un voisin coloré, on remplit le tableau par un 1 à l'indice de la couleur. Par exemple, si le premier voisin coloré est de couleur 2, le tableau ressemblera à $[0, 1, 0, \dots, 0]$. Ainsi, si le sommet a un voisin de couleur 1, il suffit simplement de parcourir le tableau jusqu'à trouver un 0. L'indice du premier 0 nous donne la plus petite couleur non utilisée par les voisins du sommet.

A l'issue de DSatur, les sessions sont créées.

Ainsi, après avoir codé le programme de DSatur, il fallait se pencher sur la question de l'assignation des créneaux ainsi que des salles d'examen. Pour cela, nous avons défini 2 fonctions "emploiDuTemps(dictionnaire)" où "dictionnaire" est le dictionnaire renvoyé plus tôt par l'algorithme de DSatur, et "assignation_salle(emploiDutemps, promotions, salles)" où "emploiDutemps" est l'emploi du temps renvoyé par la fonction "emploiDuTemps", "promotions" est la liste de l'ensemble des promotions dans l'établissement, et enfin "salles" qui est la liste des salles d'examens. Ainsi, pour le code de "emploiDuTemps(dictionnaire)", nous partirons du pseudo-code suivant:

- 1) *On regarde le nombre total nbtotale de clé de "dictionnaire". Comme nous avons 5 jours au total dans la semaine d'examen, allant de lundi à vendredi, on divise nbtotale par 5 afin de savoir combien de matières il faut mettre par jour.*
- 2) *Si le nombre de matières est supérieur ou égal à 4, on ajoute le 6ème jour samedi afin d'y mettre les matières qui restent (avoir au maximum 3 matières par jour fait partie de nos contraintes molles). Sinon on reste avec 5 jours.*

- 3) *On crée notre emploi du temps qui sera sous la forme d'un dictionnaire avec comme clé un jour de la semaine, et comme valeur la liste des matières qui sont les valeurs de "dictionnaire".*

L'implémentation de ce pseudo-code ne nous a pas posé de réels problèmes, mais nous a tout de même obligé à nous questionner sur la manière de répartir un trop grand nombre de matières. Nous avons tout d'abord pensé à ne pas prendre en compte la contrainte molle qui oblige un nombre maximal de 3 matières par jour, mais nous nous sommes finalement résolus à ajouter un 6ème jour d'épreuves si ce nombre venait à être trop grand afin de respecter cette contrainte molle. Ainsi, cet algorithme ne fonctionne que pour un nombre maximum de $3 \times 6 = 18$ matières, ce qui est un nombre raisonnable de matières pour une école comportant plusieurs promotions. En effet, pour une école comportant 4 filières différentes, cela fait un total moyen de 5 matières par filière, promotion, ce qui est tout à fait concevable et réaliste.

Pour le code de "assignation_salle(emploiutemps, promotions, salles)", nous partirons du pseudo-code suivant:

- 1) *On initialise notre heure de début de journée à heuredebut = 510 correspondant à 510 min = 8h30 du matin.*
- 2) *Pour chaque matière, on regarde le nombre total d'élèves qui passent cette matière. On regarde également combien de temps dure son examen.*
- 3) *Pour chaque salle, on regarde si elle est disponible en termes d'horaire pour une matière avec une certaine durée. Si la salle est disponible en termes d'horaire, on regarde si sa capacité est supérieure ou égale au nombre d'élèves qui passent une même matière. En fonction de sa capacité, on assigne ou non cette salle à cette matière.*
- 4) *On retourne un dictionnaire avec comme clé un jour de la semaine et comme valeur un dictionnaire avec comme clé un créneau et comme valeur la liste de matière et leurs salles assignées.*

Pour implémenter "assignation_salle(emploiutemps, promotions, salles)", nous avons rencontré des problèmes. Le premier étant celui des créneaux. Notre première idée était de scinder chacun des jours de la semaine de la façon suivante: "lundi1", "lundi2", "lundi3". "lundi1" correspondrait à lundi de 8h30 à 10h, "lundi2" correspondrait à lundi de 11h à 12h30 et enfin "lundi3" correspondrait à lundi de 13h30 à 15h. Le problème avec cette façon de penser est qu'elle enlève énormément de flexibilité à notre algorithme. En effet, cela voudrait dire que chaque examens doit durer 1h30 maximum, ce qui n'est pas du tout réaliste. Certaines matières sont plus importantes et peuvent donc durer bien plus que 1h30.

De plus, si un examen dure 1h, cela voudrait donc dire qu'il y a 30 min où la salle est vide et aucun autre examen ne peut commencer pendant ces 30 minutes. Ainsi, on a

réfléchi à une autre manière de séparer nos journées, qui s'adapterait aux durées des différents examens.

Tout d'abord, notre liste de disponibilités pour chaque salle est définie de la manière suivante, déjà expliquée page 18: [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0]. Chaque chiffre représente une session de 30 min. Un 0 signifie que la salle est disponible, et un 1 signifie que la salle est occupée, indisponible. Il y a ici un total de 19 sessions de 30 minutes. Nos journées d'examen commencent donc à 8h30 et finissent à 17h30.

Ensuite, en partant du principe que chaque examen a une durée qui est un multiple de 30, nous avons donc mis dans l'attribut "duree" de la classe "Matières" un chiffre, correspondant au multiplicateur de 30. Par exemple, si la durée de la matière "Technique_dessin" est 2, cela signifie que l'examen dure $2 * 30$ minutes = 60 minutes = 1 heure. Cela nous permettait donc de savoir combien de 0 d'affilés, de suite, nous devions chercher dans la liste de disponibilités d'une salle. Par exemple, pour "Technique_dessin", il suffit de chercher deux 0 de suite dans la liste de disponibilités d'une salle afin de lui affecter cette salle et un créneau. Il ne reste plus qu'à remplacer les deux 0 par des 1, afin de préciser que cette salle n'est plus disponible à ces horaires là. Nous avons finalement créé une variable heure_depart et une autre heure_fin, où heure_depart serait l'heure à laquelle commence la première épreuve, et heure_fin serait heure_depart auquel on ajoute la durée de l'épreuve. Cela permet au code d'être plus flexible, et de s'adapter aux durées des examens. Ainsi, après assignation d'une salle à une matière, heure_depart doit donc prendre la valeur de heure_fin. Mais une de nos contraintes molles nous a posé un petit problème. La contrainte est la suivante: "il faut 1 h de pause entre chaque examen d'une même promotion". Ainsi, cette contrainte nous a obligé à vérifier si 2 épreuves qui se déroulaient le même jour étaient des épreuves passées par une même promotion. Si c'était le cas, alors on ne pouvait pas mettre la deuxième matière tout de suite après la première épreuve. Il fallait mettre 1h de pause. Mais si ces 2 matières n'étaient pas communes à la même promotion, alors on pouvait se permettre de la mettre tout de suite après la première matière.

Un autre problème que nous avons rencontré fût les capacités des différentes salles. En effet, certaines de nos matières comme par exemple "Technique_dessin" est une matière que toutes les promotions ont en commun. Ainsi, cela représente un total de 170 élèves qui passent cette matière le même jour. Et aucune de nos salles n'a une capacité aussi grande. Ainsi, dans un cas comme celui-ci, il nous fallait répartir ces élèves de la façon suivante: on remplit une salle, et dès qu'elle est remplie, on passe à une autre salle que l'on remplit à son tour. Mais cela obligeait évidemment de trouver une salle disponible aux mêmes horaires. Enfin, un souci mineur fût celui de la conversion des horaires en des horaires plus lisibles. En effet, au début le code nous renvoyait un horaire de la forme "8.5h jusqu'à 9.5h". Ainsi, nous avons converti les "0.5" par des demi-heures.

A l'issue de ces 2 codes, nous avons notre emploi du temps final.

e. L'interface

Nous avons eu peu de jours afin de réaliser une interface graphique qui était qualifiée de bonus. Nous avons choisi de représenter notre travail sous forme d'emploi du temps qui est en fait un résumé graphique de notre code et de ses résultats. Nous avons décidé d'utiliser tkinter grâce à sa facilité d'utilisation. L'interface créée est plutôt destinée à l'administration et est composée de deux pages principales: celle d'identification et celle où est affiché l'emploi du temps. Nous avons rencontré des difficultés afin de lier ces deux pages mais finalement nous avons trouvé la solution grâce à une bonne utilisation des objets tkinter et des fonctions intégrées telles que *destroy()*, *delete()* et *insert()*. L'objet *Text* a aussi été problématique pendant un moment car nous n'arrivions pas à faire apparaître le text tandis qu'avec l'objet *Entry* tout fonctionnait. Il a fallu tout simplement revoir la configuration des objets *Text* et bien indexer les fonctions *delete()* et *insert()* notamment en écrivant 0.0 et non 0.

Après avoir construit le corps de l'interface, il fallait réfléchir à la manière de lier le code à l'interface graphique. Le plus judicieux a été de créer 5 listes avec les informations des 5 journées de la semaine d'examen. Nous voulions au départ segmenter davantage le visuel de notre emploi du temps en utilisant plus de listes par exemple et un emploi du temps par demi-journée mais la construction de ces listes était compliquée. En effet, les horaires étant très diverses et en ayant utilisé des clés, il était difficile de traiter les informations et notamment les horaires par demi-journées. Une autre difficulté rencontrée aussi a été de transformer les éléments de la liste d'une journée en chaîne de caractères. La solution finalement retenue a été de créer les variables globales afin de pouvoir traiter les nouvelles listes composées de chaînes de caractères dans la fonction *remplir()*.

Notre visuel est à améliorer ainsi que les informations affichées mais nous avons abouti à ce résultat:

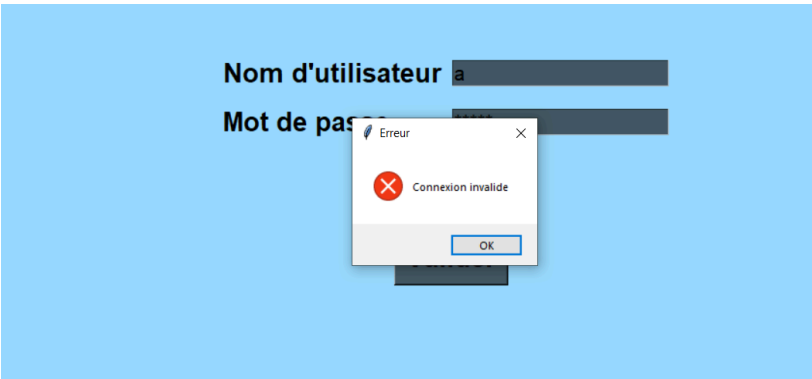
Page de connexion:

Le nom d'utilisateur est: **admin**

Le mot de passe est: **team4**



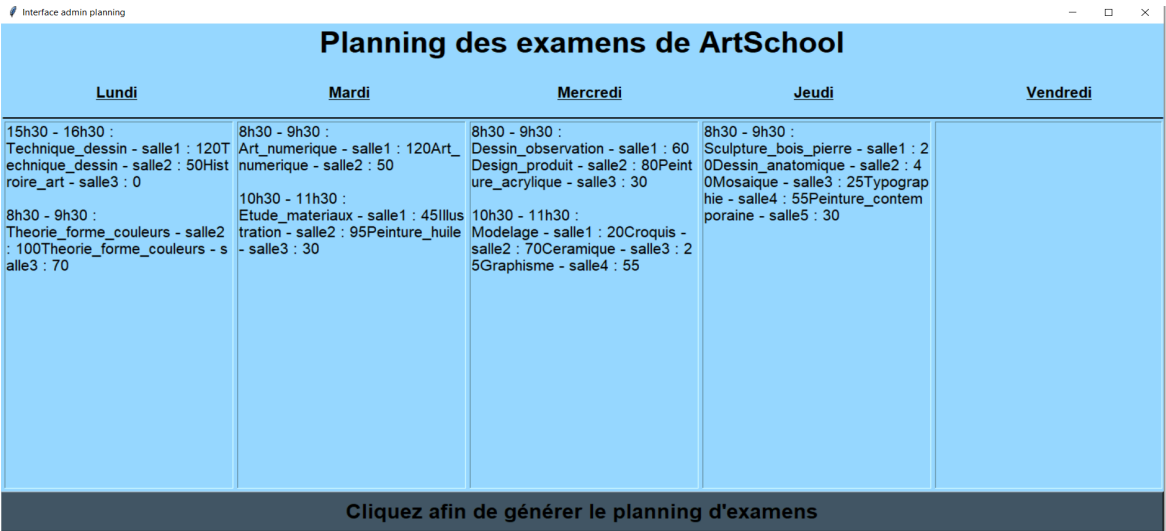
Si l'identifiant et/ou le mot de passe est incorrect ce message s'affiche:



L'emploi du temps avant d'être complété:



Et enfin l'emploi du temps après avoir été complété:



Conclusion

Nous avons d'abord réalisé un état de l'art des différentes approches permettant la planification optimale d'examen. Nos recherches se sont principalement axées sur les méthodes de coloration de graphe, et les approches heuristiques. Nous avons présenté les algorithmes DSatur, de Welsh et Powell, de Récursive Largest First et une approche métaheuristique génétique.

En prenant en compte notre cahier des charges, nous avons choisi de nous concentrer sur une approche par coloration de graphe, en utilisant DSatur. Cette méthode favorise un développement informatique rapide et nous permet de répondre aux exigences du sujet quant à la gestion des contraintes.

Nous avons ensuite implémenté la méthode DSatur auquel nous avons rajouté un algorithme d'assignation de salles et d'horaires. La phase d'implémentation fut assez difficile car nous codions tous assez différemment et la mise en commun a dû se faire très rapidement à cause des contraintes de temps.

Ainsi, ce fut un projet très intéressant et formateur mais nous aurions apprécié avoir un peu de temps supplémentaire pour pouvoir développer davantage sur le sujet, fournir un code plus robuste avec plus de possibilités d'adaptation et une interface avec un espace élève qui aurait afficher seulement l'emploi du temps de l'élève en question.

Bibliographie

[1] MALEK Maria , DUJOL Romain : Théorie des graphes – Notes de cours

[1] CHERAITIA Meryem : Planification d'examens :
<https://dspace.univ-guelma.dz/xmlui/bitstream/handle/123456789/131/these%20cheraitia%20finale%202018.pdf?sequence=1&isAllowed=y>

[2] TERNIER Ian-Christopher : Exact algorithms for the Vertex Coloring Problem and its generalisations : <https://theses.hal.science/tel-01867961>

[2] DSATUR : <https://fr.wikipedia.org/wiki/DSATUR>

[3] MOALIC Laurent : Coloration de graphe - méthodes et applications :
https://pedagogie.ac-strasbourg.fr/fileadmin/pedagogie/isn/ISN/Conferences/cours_isn_Coloration_graphe.pdf

[4] ELEN Abdullah : A complete solution to exam scheduling problem : A case study : <https://dergipark.org.tr/en/download/article-file/2025907>